

Wolfpack Strategic Doctrine Expansion – Build Specification v1

Purpose

This document expands Wolfpack's **Strategic Doctrine** into an actionable build specification. It bridges the high-level principles defined in the doctrine to concrete implementation steps that engineers, product owners and governance teams must follow during the build phase.

Recap of Core Principles

Wolfpack's mission is to build AI-native operational systems that generate durable recurring revenue with minimal human overhead and high operational stability. The strategic doctrine encompasses:

- **Core priorities:** runtime stability, governance integrity, operational clarity, recurring revenue, institutional memory, controlled adaptation, and capability expansion.
- **System doctrine:** prefer boring, reliable systems over clever architecture; place governance before automation; avoid uncontrolled production experiments; preserve rollback capability; use GitHub as the canonical source-of-truth; minimize human API/manual copy–paste workflows; preserve operational calm and simplicity; treat failures as institutional learning.
- **Behavioral requirements:** challenge weak assumptions, identify operational risks, highlight hidden support burden and vendor lock-in risks, minimize unnecessary complexity, preserve scope discipline, prefer reversible decisions, and separate production systems from experiments.
- **Execution discipline:** perform one operational step at a time; validate before expansion; avoid feature stacking during instability; require rollback paths before deployment; enforce governance review; disallow unrelated changes in the same deployment.
- **Architecture doctrine:** separate intelligence/orchestration from execution, separate governance from runtime execution, ensure vendor replaceability, design systems that continue functioning if individual tools disappear, and mandate operational memory and logging.
- **Economic doctrine:** prioritize recurring revenue, low support burden, and operational leverage; deprioritize hype-driven systems and high-touch manual businesses; optimize for compounding operational advantage.

Implementation Guidelines

This section translates strategic principles into build-phase tasks.

Core Priorities

- **Runtime stability:** choose mature, well-supported languages and frameworks; avoid unnecessary experimental dependencies; implement health checks, circuit breakers, and graceful degradation; enforce automated testing and staging environments.
- **Governance integrity:** embed approval gates in CI/CD pipelines; codify approval policies in policy-as-code repositories; track all governance decisions in the memory `decisions/` folder; implement permission boundaries for code and configuration changes.
- **Operational clarity:** document system boundaries, responsibilities, and escalation paths; generate

architectural diagrams and keep them version-controlled; standardize naming and directory structures; use dashboards and runbooks for transparency.

- **Recurring revenue:** design products and services as recurring subscriptions; instrument billing hooks early; avoid custom professional services that scale linearly with headcount; build product features that enable self-service onboarding and upsell paths.
- **Institutional memory:** adopt the memory architecture specification; require all postmortems, decisions and deployments to be captured as markdown files with metadata; implement summarization pipelines to make memory digestible; enforce tagging and provenance metadata.
- **Controlled adaptation:** treat changes as hypotheses; run experiments in sandbox environments; create feature flags for new capabilities; require documented rollback paths; schedule reviews of experimental results before graduation to production.
- **Capability expansion:** maintain a roadmap of desired capabilities with clear business justification; build modular architecture so new capabilities can be added via independent services; avoid coupling new features deeply into core runtime.

System Doctrine Execution

- **Prefer boring reliable systems:** select proven databases (e.g., PostgreSQL), messaging systems (e.g., Kafka), and orchestration platforms (e.g., Kubernetes) unless a clear advantage justifies alternatives; avoid adopting bleeding-edge technologies without support plans.
- **Governance before automation:** implement manual approval workflows before automating processes; codify policies in a `governance/` directory and enforce them via CI checks; refuse automation tasks that bypass governance checks.
- **No uncontrolled production experiments:** deploy new features behind feature flags; restrict usage to internal testers or controlled beta groups; monitor metrics and log anomalies; disable or roll back quickly if issues arise.
- **Preserve rollback capability:** maintain previous versions of deployments and configuration; implement one-click rollback scripts; ensure database migrations are reversible; document rollback steps in deployment manifests.
- **GitHub as canonical source-of-truth:** centralize infrastructure-as-code, application code, memory, and documentation in repositories; disallow ad-hoc changes via consoles or UIs; enforce pull-request workflows and code reviews.
- **Minimize human API/manual copy-paste:** build integrations with communication platforms (Slack, email) to send notifications and gather approvals; write scripts to synchronize data between systems; discourage manual data entry by humans.
- **Preserve operational calm:** implement rate limiting and back-pressure; design systems to degrade gracefully; prevent alert storms by tuning thresholds; use on-call rotations to distribute load; allocate time for maintenance and health.
- **Failures become institutional learning:** log all incidents; perform postmortems; store them in `incidents/`; add action items to the memory and track them; update specifications and runbooks based on lessons learned.

Behavioral Requirements Implementation

- **Challenge assumptions:** embed pre-mortem reviews in design processes; assign a “red team” role in design discussions to poke holes; require risk assessments before major changes.
- **Identify operational risks:** maintain a risk registry; rate each risk by likelihood and impact; tie

mitigations to specific tasks; review risks quarterly.

- **Spot hidden support burden:** track time spent on support tasks; identify sources of manual intervention; automate or refactor processes with high support load.
- **Vendor lock-in risks:** implement vendor abstraction layers (see vendor abstraction spec); avoid proprietary APIs when possible; maintain migration plans for critical vendors.
- **Minimize unnecessary complexity:** adopt simple designs; limit microservices sprawl; avoid premature generalization; regularly evaluate system complexity against business value.
- **Preserve scope discipline:** define MVP scope and freeze it; create change logs for scope additions; require justification and governance approval for expansions; track scope creep incidents.
- **Prefer reversible decisions:** choose technologies and designs that can be swapped out; avoid database engines with proprietary data formats; use feature flags for new features; document decision rationale.
- **Separate production from experiments:** maintain separate environments; enforce branch protections; label experimental components clearly; do not allow experimental code into production pipelines without review.

Execution Discipline Implementation

- **One operational step at a time:** sequence tasks in deployment pipelines; avoid bundling unrelated changes; commit small, atomic patches; require approvals for each step.
- **Validate before expansion:** establish acceptance tests and metrics for each component; deploy to staging first; perform load tests; monitor SLOs before wider rollout.
- **No feature stacking during instability:** freeze feature development during incident response; allocate cycles to stabilize before resuming new work.
- **Require rollback path before deployment:** verify rollback scripts and backups; block deployments if rollback is undefined; test rollbacks periodically.
- **Deployments require governance review:** integrate approval gates into CI; verify that required roles have signed off; record approvals in deployment manifest.
- **No unrelated changes in same deployment:** enforce commit hygiene; separate infrastructure changes from application changes; use separate pull requests for each change class.

Architecture Doctrine Implementation

- **Separate orchestration from execution:** keep high-level decision logic (Eterna) independent of execution runtime (OpenClaw); communicate via well-defined interfaces; avoid orchestrator making direct calls to vendor APIs.
- **Separate governance from execution:** store governance rules in policy engine; create governance services that control approvals; restrict execution services from bypassing governance.
- **Vendor replaceability:** build service abstraction layers; define internal contracts (see API contracts); implement adapters for each vendor; include fallbacks or alternative providers.
- **Survive tool loss:** design systems to use queues and idempotent operations; store state in persistent storage; ensure no single vendor outage can stop critical workflows.
- **Mandatory memory/logging:** ensure every service logs structured events; write to memory via approved interfaces; centralize logs and metrics (see observability spec).

Economic Doctrine Implementation

- **Recurring revenue:** design subscription billing; build features that encourage long-term use; implement account management and upselling via the AI support runtime.
- **Low support burden:** invest in AI support (see AI support runtime spec); create self-service docs; analyze common support issues; reduce manual support tasks.
- **Operational leverage:** automate deployments, scaling, and monitoring; standardize infrastructure and code templates; reuse components across services.
- **Avoid hype-driven systems:** conduct technology evaluations; avoid adopting technologies solely due to hype; align tech choices with long-term stability and cost.
- **Deprioritize high-touch manual business:** minimize professional services; focus on productized solutions; leverage AI support and automation to reduce manual interactions.
- **Optimize for compounding advantage:** invest in memory systems; capture learnings; build internal tooling to accelerate development and operations; leverage data network effects.

Conclusion

The strategic doctrine expansion translates high-level aspirations into actionable requirements. Engineering teams should use this document alongside other Wolfpack specifications (memory, event schema, API contracts, observability, etc.) to plan, design, and execute work. Governance teams must enforce these guidelines through policy frameworks and continuous review. This specification should be revisited regularly as the organization evolves.